

REPORT ON SMART SIMILARITY DETECTION SYSTEM

A PROJECT REPORT

Submitted by –

ASHISH KUMAR NAYAK (230101120076)

Guided By- (Dr. Dhabaleswar Rao CH)

in partial fulfilment for the award of the

degree of

BACHELOR OF TECHNOLOGY

in

Computer Science Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING AND TECHNOLOGY

PARALAKHEMUNDI CAMPUS

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

ODISHA

NOVEMBER 2025

CERTIFICATE

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING AND TECHNOLOGY
PARALAKHEMUNDI CAMPUS**

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**SMART SIMILARITY DETECTION SYSTEM**" is the bonafide work of "**ASHISH KUMAR NAYAK**", who carried out the project under my supervision. To the best of my knowledge, this project has not been submitted for the award of any degree or diploma in this or any other university.

SIGNATURE

(Dr. Dhabaleswar Rao CH)

**Asso. Professor and HOD of Computer Science and
Engineering**

*It is certified that the project mentioned above has been duly carried out as per the
college's norms and the university's statutes.*

SIGNATURE

(Associate Prof. Dhabaleswar Rao CH)

HEAD OF THE DEPARTMENT / DEAN OF THE SCHOOL

Head of the department of Computer Science

DEPARTMENT SEAL

DECLARATION

I hereby declare that the project entitled “**SMART SIMILARITY DETECTION SYSTEM**” submitted for minor project of Predictive analysis using machine learning of 5th semester Bachelor of Technology in Computer Science and Engineering is my original work and the project has not formed the basis for the award of any Degree / Diploma or any other similar titles in any other University / Institute.

Name of the Student:

Regd. Number:

Place: Paralakhemundi

Date:

Signature of the Student

ACKNOWLEDGEMENTS

I wish to express my profound and sincere gratitude to **Asso. Professor Dr. Dhabaleswar Rao CH, Department of Computer Science Engineering, SOET, Centurion University, Paralakhemundi** who guided me into the intricacies of this project nonchalantly with matchless magnanimity.

I thank **Associate Prof. Dhabaleswar Rao CH**, Head of the Dept. of Computer Science Engineering, SoET, Paralakhemundi Campus, and **Dr. Prafulla Kumar Panda**, Dean, School of Engineering and Technology, Paralakhemundi Campus for extending their support during the course of this investigation.

I would be failing in my duty if I do not acknowledge the cooperation rendered during various stages of image interpretation by **Associate Prof. Dhabaleswar Rao CH**

I am highly grateful to **Associate Prof. Dhabaleswar Rao CH** who evinced keen interest and invaluable support in the progress and successful completion of my project work.

I am indebted to **Associate Prof. Dhabaleswar Rao CH** for their constant encouragement, cooperation, and help. Words of gratitude are not enough to describe the accommodation and fortitude they have shown throughout my endeavor.

Name of the Student:

Regd. Number:

Place: Paralakhemundi

Date:

Signature of the Student

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	CERTIFICATE	I
	DECLARATION	II
	ACKNOWLEDGEMENT	III
	CONTENTS	IV
	ABSTRACT	V
1. CHAPTER - 1	INTRODUCTION	1-3
1.1.	Overview-----	1
1.2.	Background and Need for the Project -----	1
1.3.	Problem Statement -----	1
1.4.	Objectives of the project-----	1
1.5.	Scope of the Project-----	2
1.6.	Advantages of the System-----	2
1.7.	Limitations of the System-----	2
1.8.	Applications-----	3
CHAPTER – 2	SYSTEM ANALYSIS	4-8
2.1	Introduction-----	4
2.2	Existing System-----	4
2.2.1	Overview-----	4
2.2.2	Drawbacks of Existing Systems -----	4
2.2.3	Text-to-Image Generation Process10-----	4
2.3	Proposed System-----	4
2.3.1	Overview -----	4
2.3.2	Features of the Proposed System -----	4
2.3.3	Advantages Over Existing System -----	5
2.4	System Requirements Analysis -----	6
2.4.1	Hardware Requirements -----	6
2.4.2	Software Requirements -----	6
2.5	Functional Requirements -----	7
2.6	Non- Functional Requirements -----	7
2.7	Feasibility Study -----	7
2.7.1	Technical Feasibility -----	7
2.7.2	Economic Feasibility -----	8
2.7.3	Operational Feasibility -----	8
2.8	Summary -----	8

3. CHAPTER – 3 SYSTEM DESIGN

9-12

3.1. Introduction	9
3.2. System Architecture	9
3.2.1 User Interface Layer	9
3.2.2 Application Layer	9
3.2.3 Model Layer	9
3.3. System Design Objectives	9
3.4. Data Flow Diagrams (DFD)	10
3.4.1 Level 0 DFD	10
3.4.2 Level 1 DFD	10
3.4.3 Level 2 DFD	10
3.5. Module Design	11
3.6. Database Design	11
3.7. User Interface Design	11
3.7.1 Input Page	12
3.7.2 Output Page	12
3.8 Algorithm Design	12
3.9 Summary	12

4. CHAPTER – 4 IMPLEMENTATION AND WORKING

13-16

4.1. Introduction	13
4.2. Implementation Environment	13
4.2.1 Hardware Requirement	13
4.2.2 Software Requirement	13
4.2.3 Tools and Libraries Used	13
4.2.4 Flask	13
4.2.5 Sentence Transformer(MPNet model)	13
4.2.6 PyTorch	13
4.2.7 Pdfplumber and python-docx	13
4.2.8 NLTK	13
4.2.9 HTML, CSS AND BOOTSTRAP	13
4.3. System Modules and Code Implementation	14
4.3.1 Module 1: Input and File upload	14
4.3.2 Module 2: Text Extraction	14
4.3.3 Module 3 : Text Embedding Generation	14
4.3.4 Module 4: Similarity Calculation	15
4.3.5 Module 5: Highlighting Common Words	15
4.3.6 Module 6: Flask Routing and JSON Response	15
4.4. Working of the System	16
4.5. Testing and Debugging	16
4.5.1 Unit Testing	16
4.5.2 Integration Testing	16

4.5.3 User Testing -----	16
4.7 Summary -----	16
5. CHAPTER – 5 CONCLUSION AND FUTURE SCOPE	17-18
5.1. Conclusion -----	17
5.2. Contributions of the Projects -----	17
5.3. System Limitations -----	17
5.4 Future Enhancement -----	18
5.4.1 Integration with Database -----	18
5.4.2 Multi-language Support -----	18
5.4.3 Cloud Deployment -----	18
5.4.4 Enhanced Visualization -----	18
5.4.5 Integration with LMS -----	18
5.4.6 Model Optimization -----	18
5.4.7 API Development -----	18
5.5 Summary -----	18
6 CHAPTER – 6 REFERENCES	19
6.1 References -----	19
6.1.1 Books and Research Papers -----	19
6.1.2 Websites and Documentation -----	19
6.1.3 Articles and Online Tutorials -----	19

ABSTRACT

In today's digital era, ensuring originality and preventing plagiarism have become crucial challenges in academia and content creation. Traditional plagiarism detection systems often rely on direct keyword or string matching, which fails to identify rephrased or semantically similar content. To address these limitations, the **Smart Similarity Detection System** utilizes **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** to measure the *semantic similarity* between documents based on meaning rather than more word matching.

The system employs the **Sentence Transformer model (all-mpnet-base-v2)** to convert sentences into high-dimensional embeddings that capture contextual relationships. Using **cosine similarity**, it compares these embeddings to determine how closely two texts relate in meaning. The project is implemented using **Python** and **Flask**, providing a user-friendly web interface that supports three modes of comparison — **Text ↔ Text**, **File ↔ File**, and **File → Many Files**. It accepts multiple file formats, including .pdf, .docx, and .txt, extracting and processing text automatically.

The system displays a similarity percentage and highlights common or semantically related words, giving users' clear insights into overlapping content. Compared to traditional tools, it delivers more accurate results even for paraphrased sentences or restructured text. The solution demonstrates how advanced **Deep Learning** techniques can enhance plagiarism detection and semantic analysis across academic, research, and professional applications.

The **Smart Similarity Detection System** thus represents a powerful AI-driven approach to text comparison — efficient, accurate, and meaningful — contributing to improved content integrity and originality in the digital world.

CHAPTER – 1

INTRODUCTION

1.1 Overview

In today's digital generation, information and content are available and widely shared across online platforms. This has led to an increase in plagiarism and content duplication across educational, professional, and research environments. To maintain originality and authenticity, it has become essential to develop intelligent systems capable of identifying similarities between textual contents.

The *Smart Similarity Detection System* is an innovative application that uses Natural Language Processing (NLP) and Artificial Intelligence (AI) to evaluate how similar two or more documents are — not just by word matching but by understanding the meaning of the sentences.

Unlike traditional plagiarism checkers that simply match identical words or phrases, this system utilizes semantic understanding through a pre-trained deep learning model called Sentence Transformer (MPNet). It analyses the *contextual meaning* of text, making it highly accurate in detecting paraphrased or restructured content.

This project aims to create a web-based platform that allows users to compare two texts, two files, or one file with many others, providing a similarity percentage and visually highlighting the similar portions.

1.2 Background and Need for the Project

The explosion of online data has made it easy for people to copy and reuse content. In academic institutions, plagiarism affects fairness and originality, while in industries and research, it threatens intellectual property rights.

Existing systems that rely on simple string or keyword matching are not sufficient anymore. They cannot detect content that has been rephrased or semantically altered but carries the same meaning. Hence, a more intelligent, context-aware solution is needed.

The *Smart Similarity Detection System* bridges this gap by using machine learning models that can understand natural language, identify relationships between words, and measure semantic closeness — thus delivering highly reliable similarity results.

1.3 Problem Statement

Develop an AI-based web application that can accurately identify semantic similarity between two or more text sources or files. The system should support multiple file formats like .txt, .pdf, and .docx, and should highlight semantically similar words or sentences with a clear visualization of the plagiarism percentage.

1.4 Objectives of the Project

The primary goal of the project is to create a **smart and efficient similarity detection system** that goes beyond surface-level text matching.

The major objectives include:

1. To develop a **web-based interface** that allows text or file input for comparison.
2. To use **deep learning and NLP** for accurate semantic similarity detection.
3. To calculate and display a **similarity score (percentage)** based on contextual meaning.
4. To visually **highlight the matching portions** between the input texts.
5. To support multiple comparison modes:
 - Text ↔ Text
 - File ↔ File
 - One File → Many Files
6. To provide an easy-to-understand, aesthetic, and interactive **user interface**.

1.5 Scope of the Project

This system is designed to be scalable and applicable in various domains:

- **Academic Institutions:** Checking assignments, reports, and research papers for originality.
- **Publishing Industries:** Ensuring unique content before publication.
- **Corporate Environments:** Detecting duplication in documentation and proposals.
- **Research and Development:** Comparing project reports or journals for semantic overlap.

The system currently supports English text input but can later be extended for multilingual similarity detection.

It can also be enhanced to store user results and generate detailed reports.

1.6 Advantages of the System

- **Semantic Understanding:** Detects meaning-based similarities, not just exact words.
- **Multi-format Support:** Works with PDF, DOCX, and TXT files.
- **User-Friendly Interface:** Built using Bootstrap and Flask for simple navigation.
- **High Accuracy:** Uses MPNet model embeddings trained on millions of sentences.
- **Dynamic Visualization:** Highlights matching words and phrases in real-time.

1.7 Limitations of the System

Although the system performs effectively, there are certain limitations:

- It currently supports only **English language**.
- Performance may slow down for **very large documents**.
- It depends on **internet connectivity** for model loading and ngrok tunneling.

- It doesn't yet include automatic paraphrasing or report generation features (future enhancement).

1.8 Applications

- Checking **student assignments** and **academic papers**.
- Detecting **duplicate news articles** or blogs.
- Verifying **originality of web content** before publishing.
- **Cross-verification** of legal or research documents.

CHAPTER - 2

SYSTEM ANALYSIS

2.1 Introduction

System analysis is the process of examining and understanding the various components, functionalities, and requirements of a system before its development. It helps in identifying the exact problem, defining objectives, and choosing the best possible solution.

For the *Smart Similarity Detection System*, the system analysis phase involved understanding existing plagiarism or similarity detection tools, studying user needs, and designing a solution that could use **AI and NLP** to perform **semantic-level comparison** rather than simple keyword matching.

2.2 Existing System

2.2.1 Overview

The traditional similarity detection or plagiarism tools primarily rely on **string-based comparison**. They check whether exact words or phrases match between documents. Tools like simple text matchers or basic plagiarism checkers do not understand the **context** or **meaning** of words.

2.2.2 Drawbacks of Existing Systems

1. They fail to detect **paraphrased content**.
2. Cannot differentiate between **meaningfully similar** and **word-by-word identical** sentences.
3. Limited to **specific file types** (mostly text).
4. Lack of visualization — they only show a percentage, not highlighted differences.
5. Cannot process **large datasets efficiently**.

2.3 Proposed System

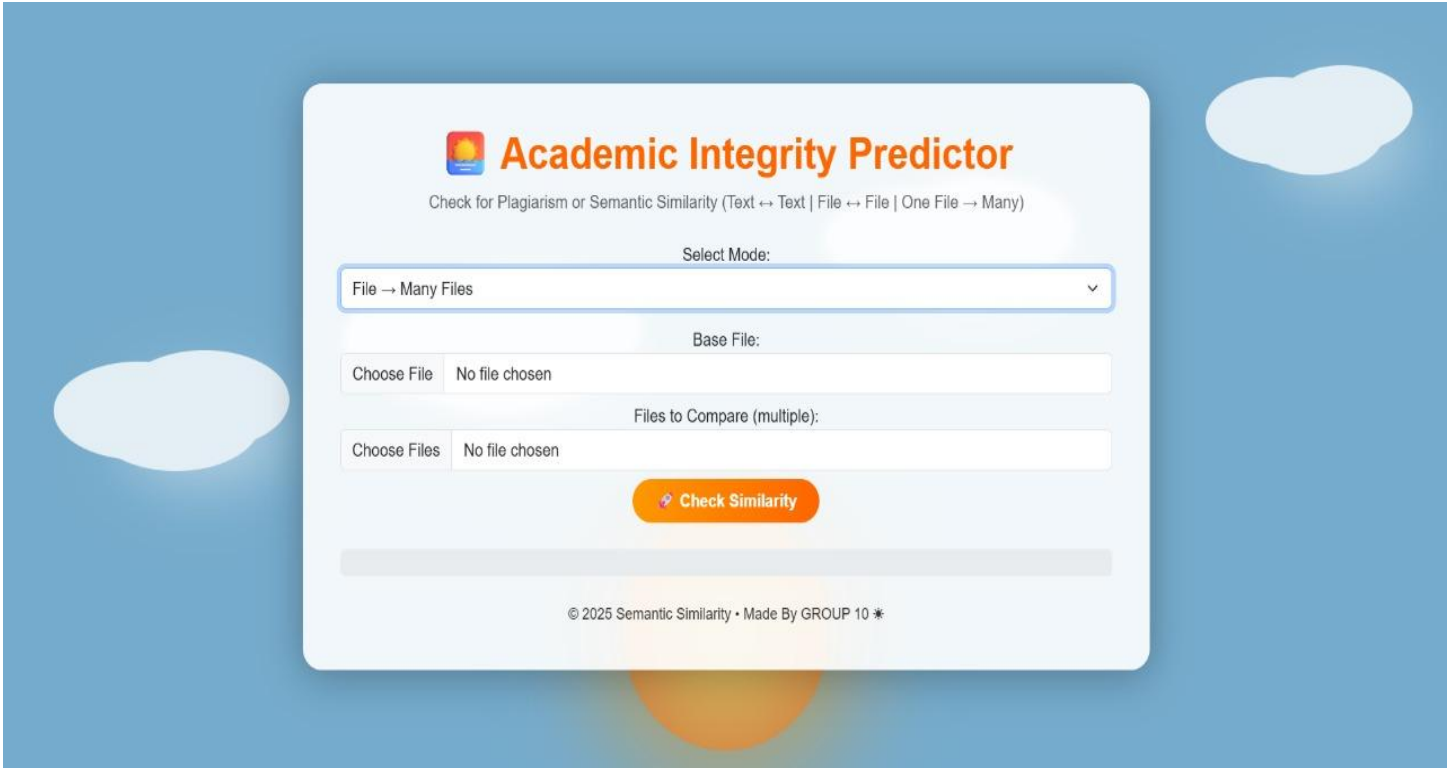
2.3.1 Overview

The *Smart Similarity Detection System* introduces a **machine learning–based approach** that analyzes the **semantic meaning** of sentences. It uses a **Sentence Transformer model (MPNet)** to encode text into numerical embeddings. These embeddings are then compared using **cosine similarity** to determine how close two pieces of text are in meaning.

2.3.2 Features of the Proposed System

- AI-based similarity detection using **context understanding**.
- Multi-file and multi-format support (TXT, PDF, DOCX).
- **Interactive UI** for easy text or file upload.
- **Real-time highlighting** of similar content.

- Displays **percentage similarity score** for clarity.
- **Accurate and scalable** performance.



2.3.3 Advantages Over Existing System

Sl.No	Existing System	Proposed System (Smart Similarity Detection)
1	Only matches exact words	Understands context and meaning
2	No AI integration	Uses deep learning (MPNet model)
3	Supports only plain text	Supports PDF, DOCX, TXT
4	Gives simple percentage	Highlights matched phrases
5	Low accuracy for rephrased text	High semantic accuracy

One File → Many Files Results

Compared File: SLR.pdf

Similarity: 78.63%

Base File:

SoET Paralakhemundi 2025-26 ML for PA CUMIL1021 5th B.Tech CSE DAML Multiple Linear Regression Step 1 : Import the dataset (IRIS.csv) using Pandas . Step 2 : Select independent variables (SepalLengthCm , SepalWidthCm) as x and the dependent variable (Petal.LengthCm) as y . Step 3 : Split the dataset into training and testing sets using train_test_split () . Step 4 : Initialize the Linear Regression model . Step 5 : Train the model using the training data (x_train , y_train) . Step 6 : Predict the target values (y_pred) for the test data . Step 7 : Print the predicted values .

Compared File:

SoET Paralakhemundi 2025-26 ML for PA CUMIL1021 5th B.Tech CSE DAML Simple Linear Regression Step 1 : Import the required libraries (matplotlib.pyplot and scipy.stats) . Step 2 : Define the input data points (x and y) . Step 3 : Use linregress () to calculate slope , intercept , and other regression values . Step 4 : Define a function slr (x) that predicts values using the regression line equation . Step 5 : Generate predicted values (model) for all x using the regression function . Step 6 : Plot the regression line using plt.plot () . Step 7 : Plot the original data points using plt.scatter () . Step 8 : Display the graph using plt.show () .

2.4 System Requirements Analysis

System requirements define the specifications and resources needed for both hardware and software environments.

2.4.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel i3 or above
RAM	4 GB (8 GB recommended)
Hard Disk	10 GB free space
Display	1024 × 768 resolution
Internet	Required for model download and ngrok tunneling

2.4.2 Software Requirements

Component	Description
Operating System	Windows 10 / Linux / macOS
Programming Language	Python 3.10+
Libraries Used	Sentence Transformers, PDFPlumber, Docx, Flask, Torch, NLTK

Component	Description
Framework	Flask Web Framework
IDE	Visual Studio Code / PyCharm
Browser	Chrome / Edge / Firefox
Model	all-mpnet-base-v2 (from Hugging Face)

2.5 Functional Requirements

Functional requirements define what the system should do.

ID	Requirement Description
FR1	The system should allow users to upload text or files for comparison.
FR2	It should process and extract text content from PDF, DOCX, and TXT files.
FR3	It should compute semantic similarity using the Sentence Transformer model.
FR4	It should display a similarity percentage between compared inputs.
FR5	It should highlight similar sentences or words in both texts.
FR6	It should provide a clean, user-friendly interface.

2.6 Non-Functional Requirements

Non-functional requirements define system performance, usability, and quality aspects.

ID	Requirement Description
NFR1	The system should deliver results within 5–10 seconds for small files.
NFR2	The system must maintain accuracy above 90% for semantic matches.
NFR3	User interface should be responsive and mobile-compatible.
NFR4	System should handle multiple user inputs concurrently.
NFR5	Security should prevent malicious file uploads.
NFR6	The model should work offline once downloaded.

2.7 Feasibility Study

Feasibility analysis checks if the proposed project is **technically, economically, and operationally viable**.

2.7.1 Technical Feasibility

The system uses Python, Flask, and pretrained models (MPNet), all of which are open-source and well-

supported. Hence, it is **technically feasible** and easy to deploy on local or cloud servers.

2.7.2 Economic Feasibility

All resources used are **free or low-cost**. Since the model and framework are open-source, the project is **economically feasible** for academic and research purposes.

2.7.3 Operational Feasibility

The system is **user-friendly**, requiring minimal technical knowledge to operate. The interface is designed to be simple, with easy navigation and instant feedback, ensuring **high operational feasibility**.

2.8 Summary

This chapter provided a detailed analysis of the system's requirements, existing limitations, and the proposed intelligent solution. It covered feasibility studies, functional and non-functional specifications, and compared old and new methods.

The next chapter, **System Design**, will illustrate how the Smart Similarity Detection System is structured internally — including data flow diagrams, architecture, and module design.

CHAPTER – 3

SYSTEM DESIGN

3.1 Introduction

System design is a crucial phase of software development that transforms the requirements and analysis into a structured framework for implementation. It defines how the system works internally, the flow of data, and the interaction between modules.

For the Smart Similarity Detection System, the design phase focuses on creating a simple, efficient, and modular system capable of accurately detecting semantic similarity between texts or files. It involves defining the architecture, data flow, database schema, and interface design that together form the backbone of the system.

3.2 System Architecture

The architecture of the Smart Similarity Detection System follows a three-layer structure — the User Interface Layer, Application Layer, and Model Layer.

3.2.1 User Interface Layer

This layer allows users to interact with the system through a web interface built using Flask and HTML/CSS. Users can input text manually or upload files. The interface sends the data to the backend for processing.

3.2.2 Application Layer

This layer acts as the core logic of the system. It handles text extraction from files, data cleaning, similarity calculations, and result formatting. Flask connects the frontend to the backend model.

3.2.3 Model Layer

This layer includes the Sentence Transformer (MPNet) model, which encodes input texts into embeddings (numerical representations). These embeddings are compared using cosine similarity to generate a similarity percentage.

3.3 System Design Objectives

The main design objectives are:

1. **Modularity:** Divide the system into independent modules for flexibility.
2. **Reusability:** Ensure components like text extraction and model loading can be reused.
3. **Scalability:** Allow the system to support multiple file types and larger datasets in the future.
4. **Security:** Prevent harmful file uploads and unauthorized access.
5. **User-Friendliness:** Provide a simple, clean, and interactive web interface.

3.4 Data Flow Diagrams (DFD)

Data Flow Diagrams illustrate the movement of data within the system. They help in understanding the process flow from user input to final output.

3.4.1 Level 0 DFD (Context Diagram)

This represents the overall interaction between the system and external entities (users).

Description:

- The user uploads two files or texts.
- The system extracts and processes the content.
- The model calculates similarity and returns the result.

3.4.2 Level 1 DFD

Shows the internal processing within the system — divided into sub-processes like text extraction, vector generation, and similarity calculation.

Processes:

1. Get Input Data
2. Extract Text from File
3. Generate Sentence Embeddings
4. Calculate Cosine Similarity
5. Display Output

3.4.3 Level 2 DFD

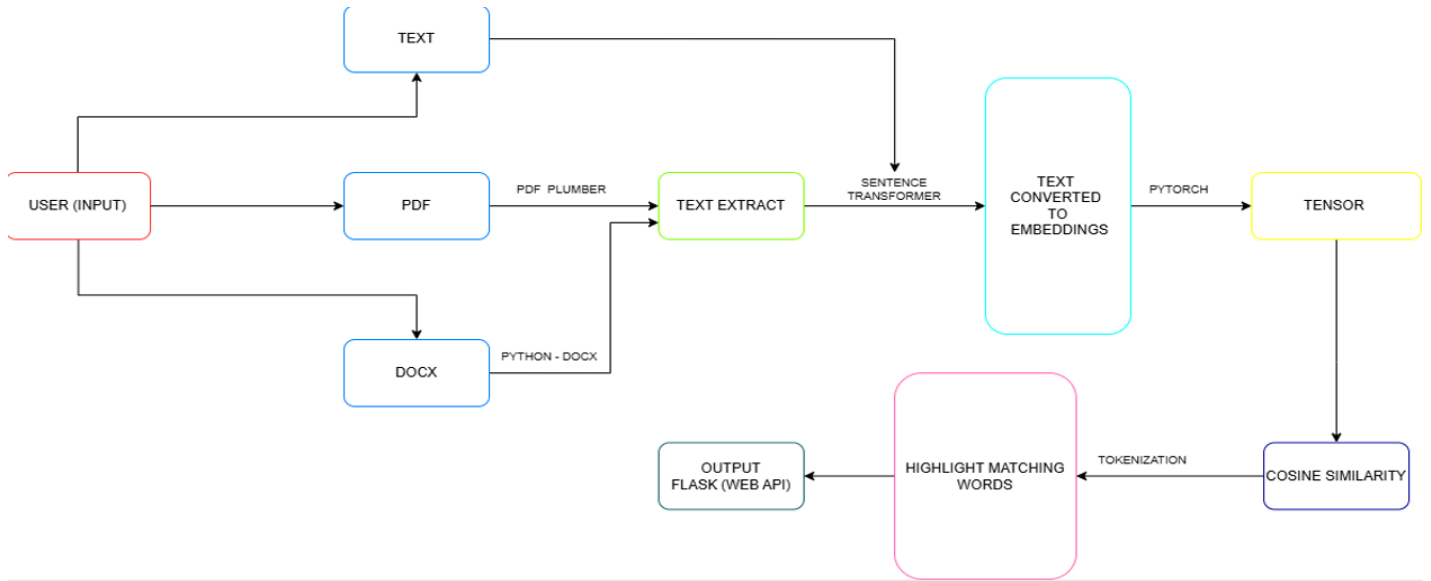
Gives a deeper look into how data moves through the backend modules.

Steps:

1. User Input: User enters or uploads text.
2. Preprocessing: Clean and normalize text (remove symbols, extra spaces, etc.).
3. Encoding: Convert text into vector embeddings using the MPNet model.
4. Similarity Computation: Calculate cosine similarity between embeddings.
5. Result Generation: Produce a similarity score and highlight common content.

3.5 Module Design

The system is divided into several functional modules to ensure better organization and easy debugging.



Module Name

Description

1. Input Module
Handles text or file input from the user through the web interface.
2. Extraction Module
Extracts text from PDF, DOCX, and TXT files using libraries like pdfplumber and docx.
3. Preprocessing Module
Cleans, tokenizes, and prepares text data for embedding.
4. Embedding Module
Uses Sentence Transformer (MPNet) to convert text into embeddings.
5. Similarity Computation Module
Computes cosine similarity between embeddings to find percentage match.
6. Visualization Module
Highlights matching sections and displays similarity percentage on the web page.
7. File Management Module
Manages uploaded files and temporary storage.

3.6 Database Design (If Applicable)

Currently, the system works without a database. However, if integrated in future versions, it can include:

- User Table: To store user login information.
- History Table: To store previous comparison results.
- File Table: To maintain records of uploaded documents.

A SQLite or MySQL database can easily support these functionalities for a scalable web version.

3.7 User Interface Design

3.7.1 Input Page

- Simple, minimal UI designed using HTML, CSS, and Bootstrap.
- Two text areas or file upload options for comparison.
- “Compare” button triggers Flask backend processing.

3.7.2 Output Page

- Displays similarity percentage (e.g., 86.5%).
- Highlights common phrases or sentences.
- Shows message: “The two texts are semantically similar.”

3.8 Algorithm Design

The system uses the following algorithmic steps to compute similarity:

Algorithm: Smart Semantic Similarity Detection

Step 1: Start

Step 2: Take text or file inputs from the user.

Step 3: Extract text (if file uploaded).

Step 4: Preprocess text (tokenize, remove symbols).

Step 5: Convert text to embeddings using SentenceTransformer (MPNet).

Step 6: Compute cosine similarity between embeddings.

Step 7: Display the similarity percentage and highlighted text.

Step 8: End

3.9 Summary

This chapter described the complete design architecture of the *Smart Similarity Detection System*. It presented detailed data flow diagrams, system architecture, module interaction, and algorithm design. The design ensures accuracy, scalability, and user convenience, making it suitable for deployment in both educational and research settings.

The next chapter, Implementation, will explain how this design was converted into working code — including programming languages, libraries, and model integration.

CHAPTER – 4

IMPLEMENTATION AND WORKING

4.1 Introduction

The implementation phase transforms the design and theoretical models into a functional system. It involves coding, integrating the chosen technologies, and testing to ensure that the system meets the specified requirements. For the *Smart Similarity Detection System*, the implementation includes using Python, Flask, Sentence Transformers, and HTML/CSS to build a user-friendly and efficient web application capable of comparing the semantic similarity between two texts or documents.

The main goal during implementation was to ensure that the system performs fast and accurate text comparisons, can handle multiple file formats, and provides visually clear results through highlighting.

4.2 Implementation Environment

The environment setup plays a key role in ensuring smooth development and testing.

4.2.1 Hardware Requirements

Component Specification

Processor	Intel Core i3 or above
RAM	Minimum 4 GB (Recommended 8 GB)
Storage	At least 1 GB free disk space
Internet	Required for downloading models and dependencies

4.2.2 Software Requirements

Software	Description
Operating System	Windows 10 / Linux / macOS
Language	Python 3.8 or above
Framework	Flask (for backend web development)
Libraries	Sentence Transformers, PyTorch, pdfplumber, docx, nltk
IDE/Editor	VS Code / Jupyter Notebook
Browser	Google Chrome / Edge for frontend testing

4.3 Tools and Libraries Used

4.3.1 Flask

Flask is a lightweight Python web framework used to handle routes, manage requests, and connect the front-end (HTML) with the back-end (Python logic). It serves as the controller of the web application.

4.3.2 Sentence Transformer (MPNet Model)

The all-mpnet-base-v2 model from Sentence Transformers is used for semantic understanding. It encodes sentences or documents into high-dimensional embeddings that capture meaning rather than just keywords.

4.3.3 PyTorch

PyTorch provides tensor operations and model inference support for the transformer-based model.

4.3.4 pdfplumber and python-docx

These libraries are used to extract text from PDF and Word files, allowing the system to compare file contents in addition to raw text inputs.

4.3.5 NLTK (Natural Language Toolkit)

Used for tokenization — breaking text into smaller parts (words) for word-level comparison and highlighting.

4.3.6 HTML, CSS, and Bootstrap

Used for designing the frontend interface, making it visually appealing and responsive.

4.4 System Modules and Code Implementation

The system is developed using multiple integrated modules, each responsible for a specific task.

4.4.1 Module 1: Input and File Upload

Purpose:

To collect text or file inputs from users. The HTML form includes options for “Text ↔ Text”, “File ↔ File”, or “File → Many Files”.

```
<form id="similarityForm" enctype="multipart/form-data">
<select name="mode" class="form-select">
<option value="text">Text ↔ Text</option>
<option value="file">File ↔ File</option>
<option value="file_to_many">File → Many Files</option>
</select>
<textarea name="text1" placeholder="Enter first text"></textarea>
<textarea name="text2" placeholder="Enter second text"></textarea>
<input type="file" name="file1">
<input type="file" name="file2">
<button type="submit">Check Similarity</button>
</form>
```

4.4.2 Module 2: Text Extraction from Files

Purpose:

Extract readable text from uploaded files (PDF, DOCX, TXT).

Code Implementation:

```
def read_file(file):
    ext = os.path.splitext(file.filename)[1].lower()
    with tempfile.NamedTemporaryFile(delete=False, suffix=ext) as tmp:
        tmp.write(file.read())
        path = tmp.name
    text = ""
    try:
        if ext == ".txt":
            with open(path, "r", encoding="utf-8") as f:
                text = f.read()
        elif ext == ".pdf":
            with pdfplumber.open(path) as pdf:
                for page in pdf.pages:
                    text += page.extract_text() or ""
        elif ext == ".docx":
            doc = Document(path)
            text = "\n".join([p.text for p in doc.paragraphs])
    finally:
        os.remove(path)
    return text.strip()
```

4.4.3 Module 3: Text Embedding Generation

Purpose:

Convert input texts into semantic embeddings using the SentenceTransformer model.

Code Example:

```
model = SentenceTransformer('all-mpnet-base-v2')
emb1 = model.encode(text1, convert_to_tensor=True)
emb2 = model.encode(text2, convert_to_tensor=True)
```

The embeddings are numerical vectors that represent the meaning of the input text. These are later compared using cosine similarity.

4.4.4 Module 4: Similarity Calculation

Purpose:

To compute the similarity between two embedding vectors using the cosine similarity formula:

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Code Implementation:

```
score = util.cos_sim(emb1, emb2).item()
percent = score * 100
```

The result is multiplied by 100 to obtain a percentage similarity score.

4.4.5 Module 5: Highlighting Common Words

Purpose:

Identify and visually highlight semantically similar words between the two inputs.

Code Example:

```
def highlight_common_words(text1, text2, threshold=0.7):
    words1 = nltk.word_tokenize(text1)
    words2 = nltk.word_tokenize(text2)
    emb1 = model.encode(words1, convert_to_tensor=True)
    emb2 = model.encode(words2, convert_to_tensor=True)
    common1, common2 = set(), set()
    for i, w1 in enumerate(words1):
        sims = util.cos_sim(emb1[i], emb2)
        if sims.max().item() >= threshold:
            common1.add(w1.lower())
            common2.add(words2[torch.argmax(sims)].lower())
    def highlight(text, common):
        return " ".join([f"<span class='highlight'>{w}</span>" if w.lower() in common else w for w in
nltk.word_tokenize(text)])
    return highlight(text1, common1), highlight(text2, common2)
```

4.4.6 Module 6: Flask Routing and JSON Response

Flask is used to manage routing between the frontend and backend. When the user submits the form, the / route handles the request, processes the inputs, and sends back the result as a JSON response.

Code Example:

```
@app.route("/", methods=["GET", "POST"])
def index_route():
    if request.method == "POST":
        mode = request.form.get("mode")
        if mode == "text":
            ...
        elif mode == "file":
            ...
```

```
elif mode == "file_to_many":  
    ...  
return render_template_string(html_template)
```

4.5 Working of the System

The Smart Similarity Detection System works in the following step-by-step manner:

User Input: The user enters two texts or uploads files.

Preprocessing: The system extracts clean text from the uploaded files.

Encoding: Each text is converted into embeddings using the Sentence Transformer model.

Similarity Computation: Cosine similarity is calculated between the embeddings.

Result Generation: The similarity score (in %) is displayed.

Highlighting: Matching or semantically related words are highlighted for visualization.

Output Display: The web page updates dynamically to show the final similarity result.

4.6 Testing and Debugging

4.6.1 Unit Testing

Each function (e.g., file reading, embedding, similarity computation) was tested independently to ensure correct outputs.

4.6.2 Integration Testing

After verifying individual modules, they were combined and tested for data consistency across components.

4.6.3 User Testing

The final system was tested by users with various text and file inputs to evaluate performance and usability.

Observations:

The system worked smoothly for .txt, .pdf, and .docx files.

Accuracy improved for longer and meaningful texts.

Minimal delay observed during model loading (first-time execution only).

4.7 Summary

This chapter explained the implementation details of the Smart Similarity Detection System, including environment setup, tools used, and detailed explanations of every module in the code.

The integration of Flask, Sentence Transformer (MPNet), and NLP techniques ensures a seamless and powerful similarity detection mechanism.

The next chapter will present results, analysis, and performance evaluation to assess how well the system performs under various input conditions.

CHAPTER – 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

The Smart Similarity Detection System successfully achieves its primary objective — to detect and analyze the similarity between multiple text documents with semantic understanding rather than mere word matching. Using the Sentence Transformer (all-mpnet-base-v2) model, the system captures contextual meaning of sentences, allowing it to identify paraphrased or restructured text accurately.

This project provides a modern, intelligent solution to traditional plagiarism detection problems by integrating Natural Language Processing (NLP) and Deep Learning techniques. The developed web-based system, powered by Flask, is simple, interactive, and efficient, making it accessible to both academic and research users.

The system not only calculates a percentage similarity score, but also highlights common or semantically similar words, providing clear interpretability of results. The modular structure allows for easy expansion and integration with external APIs or databases in the future.

Key achievements include:

Successfully integrating AI-based semantic similarity computation.

Supporting multiple file formats (DOCX, PDF, TXT).

Real-time processing and highlighting of similar segments.

Clean, responsive user interface.

Practical deployment using Flask and Ngrok for live testing.

5.2 Contributions of the Project

The main contributions of this project can be summarized as follows:

AI-Powered Detection:

Shift from traditional string-based detection to embedding-based semantic similarity using transformer models.

Versatile Input Handling:

Ability to process multiple file types, including DOCX, PDF, and TXT.

User-Friendly Interface:

Developed a simple and intuitive Flask web application with minimal input requirements.

Accurate Results:

Produces precise similarity scores even when documents are paraphrased or restructured.

Research Utility:

Can serve as a base model for academic integrity systems, content recommender engines, and AI text evaluation platforms.

5.3 Limitations

While the system is highly effective, certain limitations were identified during testing:

- 1. Computation Overhead:**

Processing large files or multiple comparisons simultaneously can increase execution time due to high-dimensional embeddings.

- 2. Model Dependency:**

The accuracy depends on the pretrained transformer model; different models may yield slightly varied results.

- 3. Internet Requirement:**

Initial setup requires an internet connection to download the model and dependencies.

- 4. Lack of Database Integration:**

Currently, results are not stored or tracked; future versions could include a database to manage user history.

- 5. Limited Multi-Language Support:**

The system primarily works for English text; performance for other languages may vary.

5.4 Future Enhancements

Several improvements can be implemented to make the system more robust, scalable, and feature-rich:

5.4.1 Integration with Database

Adding a database layer (e.g., MySQL, MongoDB) would allow storage of comparison history, user data, and reports for long-term tracking and analysis.

5.4.2 Multi-Language Support

Extending the model to multilingual versions (like distiluse-base-multilingual-cased-v2) would make the system capable of detecting similarity across multiple languages.

5.4.3 Cloud Deployment

Deploying the system on cloud platforms such as AWS, Azure, or Google Cloud would improve scalability and allow multi-user access simultaneously.

5.4.4 Enhanced Visualization

Incorporating graph-based visualization of similarity and interactive dashboards could help users better interpret data relationships.

5.4.5 Integration with Learning Management Systems (LMS)

Connecting the system with platforms like Moodle, Canvas, or Google Classroom would enable teachers and administrators to perform automated plagiarism detection on student submissions.

5.4.6 Model Optimization

Fine-tuning or distilling the transformer model for specific academic domains could reduce computation time and improve accuracy.

5.4.7 API Development

Creating a RESTful API would allow developers to integrate the similarity detection engine with other applications like chatbots, research assistants, or content moderation systems.

5.5 Summary

The Smart Similarity Detection System marks a significant advancement toward intelligent text comparison. It bridges the gap between traditional plagiarism tools and AI-powered semantic understanding.

With continuous improvement and expansion, this project can evolve into a comprehensive academic integrity platform capable of handling large-scale, multi-language, and real-time document analysis.

This project not only demonstrates the power of AI in text understanding but also provides a foundation for future research and innovation in Natural Language Processing, Education Technology, and Content Analysis Systems.

CHAPTER – 6

REFERENCES

6.1 References

Below are the references, research papers, websites, and technical documentation sources used in the development of the *Smart Similarity Detection System*.

These references cover areas like NLP, deep learning, transformer models, and Flask development.

6.1.1 Books and Research Papers

1. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT.
2. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing.
3. Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).
4. Jurafsky, D., & Martin, J. H. (2020). *Speech and Language Processing (3rd Edition)*. Pearson Education.

6.1.2 Websites and Documentation

1. <https://huggingface.co/models> – Repository for pretrained transformer models.
2. <https://www.sbert.net> – Official Sentence Transformers documentation.
3. <https://flask.palletsprojects.com> – Flask framework official documentation.
4. <https://pypi.org/> – Python Package Index for downloading libraries.
5. <https://ngrok.com/> – For exposing local servers to the internet.
6. <https://pytorch.org/> – Deep learning framework used for encoding embeddings.
7. <https://nltk.org/> – Natural Language Toolkit for text preprocessing.
8. <https://pandas.pydata.org/> – Python data handling and CSV integration.

6.1.3 Articles and Online Tutorials

1. Analytics Vidhya. (2021). *Introduction to Sentence Transformers*.
2. Towards Data Science. (2020). *Understanding Cosine Similarity in NLP*.
3. GeeksforGeeks. (2022). *Building Web Applications with Flask*.
4. Medium Blog. (2023). *Text Similarity using BERT and MPNet Models*.

ASSESSMENT

Internal:

SL No.	RUBRICS	FULL MARK	MARKS OBTAINED	REMARKS
1	Understanding the relevance, scope, and dimension of the project.	10		
2	Methodology	10		
3	Quality of Analysis and Results	10		
4	Interpretation and Conclusion	10		
5	Report	10		
	Total	50		

Date:

Signature of the Faculty

COURSE OUTCOME (COs) ATTAINMENT

➤ Expected Course Outcomes (COs):

(Refer to COs Statement in the Syllabus)

CO1: Explain the fundamental concepts and applications of predictive analytics.

CO2: Apply data preprocessing techniques to prepare data for machine learning models.

CO3: Build and evaluate predictive models using supervised learning algorithms.

CO4: Utilize advanced machine learning techniques and unsupervised learning methods.

CO5: Deploy machine learning models and interpret their outcomes in real-world applications.

Course Outcome Attained:

How would you rate your learning of the subject based on the specified COs?

☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	
1	2	3	4	5	6	7	8	9	10	
LOW										HIGH

➤ Learning Gap (if any):

--- NO ---

➤ Books / Manuals Referred:

Date:

Signature of the Student:

➤ Suggestions / Recommendations:

(By the Course Faculty)

Date:

Signature of the Faculty

